

DPS912 – Assignment 2

Campus Help Desk Ticketing System Using Socket and FIFO IPC

Focus: client-server architecture, socket programming, select(), FIFO notification, protocol design, and IPC comparison

Course	DPS912 – UNIX/Linux Systems Programming
Assignment	Assignment 2
Weight	10% of final grade, adjustable by instructor
Due date	July 19, 2026
Submission	Source code, Makefile, screenshots, README/design notes, and demonstration video

1. Business Case

The college IT Help Desk needs a lightweight command-line ticketing system. Students and staff should be able to submit technical support requests from client terminals. A central server should receive requests, store tickets, respond to clients, and publish real-time activity notifications to a monitoring program.

This assignment focuses on IPC in a client-server architecture. You will build the main request/response communication using sockets, and you will use a FIFO as a separate notification channel. The goal is to understand why different IPC mechanisms fit different communication patterns.

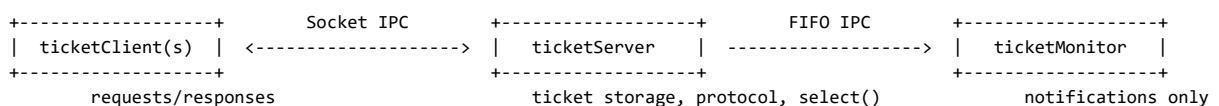
2. Learning Outcomes

- Design and implement a socket-based client-server application in C or C++ on Linux.
- Choose and justify an appropriate socket model for the system, such as AF_INET TCP or AF_UNIX stream sockets.
- Use select() to monitor multiple file descriptors, including the listening socket, connected clients, and STDIN for server commands.
- Design a simple text-based application protocol for client/server communication.
- Use FIFO/named pipe IPC for a one-way notification stream from the server to a monitor process.
- Handle graceful shutdown, cleanup, invalid input, disconnected clients, and temporary IPC files.
- Compare socket and FIFO IPC based on real-world client-server design needs.

3. System Overview

You will implement a small help desk ticketing system with three main programs:

Program	Role	IPC Used
ticketServer	Central server that accepts client commands, stores tickets, responds to clients, and writes event notifications.	Socket + FIFO
ticketClient	Command-line client used by students/staff to submit, list, close, and query tickets.	Socket
ticketMonitor	Local monitoring utility that displays server ticket events in real time.	FIFO



4. Required Socket Model

You may choose the socket model that you believe is most suitable for this client-server system. Your choice must be explained in your README and demonstration video.

5. Functional Requirements – Socket Ticket System

5.1 Client Commands

The client must allow users to send the following commands to the server:

Command	Purpose	Example
SUBMIT	Create a new help desk ticket.	SUBMIT Ali NETWORK HIGH Cannot access Linux VM
LIST	List tickets by status.	LIST OPEN or LIST ALL
DETAIL	Display one ticket by ticket ID.	DETAIL 3
CLOSE	Close an existing ticket.	CLOSE 3
STATS	Show ticket statistics.	STATS
QUIT	Disconnect this client cleanly.	QUIT

5.2 Ticket Data

Each ticket must store at least the following information:

- ticket_id – a unique positive integer generated by the server
- name – submitter name
- category – for example NETWORK, SOFTWARE, HARDWARE, ACCOUNT, OTHER
- priority – LOW, MEDIUM, HIGH, or CRITICAL
- description – short issue description
- status – OPEN or CLOSED
- created_at – timestamp or readable creation time
- closed_at – timestamp or N/A if still open

5.3 Protocol Format

You must define a simple and consistent text protocol. A pipe-separated protocol is recommended:

Client request examples:

```
SUBMIT|Ali|NETWORK|HIGH|Cannot access Linux VM
LIST|OPEN
DETAIL|2
CLOSE|2
STATS
QUIT
```

Server response examples:

```
OK|Ticket 1 created
OK|Ticket 2 closed
ERROR|Ticket not found
DATA|1|OPEN|Ali|NETWORK|HIGH|Cannot access Linux VM
```

Your server must reject invalid or incomplete commands with a clear ERROR response rather than crashing.

6. Required Use of select()

Your socket server must use `select()` to manage multiple sources of input. This is a core requirement of the assignment.

- Monitor the listening socket so new clients can connect.
- Monitor all connected client sockets so multiple clients can send commands.
- Monitor STDIN so the server operator can type runtime commands without stopping the server.
- Remove disconnected clients from the active descriptor set.
- Continue running until the server receives a shutdown command or SIGINT.

Required server-side STDIN commands:

Server Command	Expected Behaviour
show	Display all tickets currently stored in memory.
stats	Display total, open, closed, and connected-client counts.
clients	Display number of currently connected clients.
save	Write current tickets to the persistence file.
quit	Perform controlled shutdown and cleanup.

7. FIFO Notification Component

The FIFO component must be implemented as a one-way notification channel from the ticket server to a local monitor program.

- The server creates or opens a named FIFO under `/tmp`.
- The FIFO name must include the current username, for example `/tmp/ticket_notifications_<username>.fifo`.
- The server writes a notification whenever a ticket is created, closed, or when the server is shutting down.
- `ticketMonitor` reads from the FIFO and displays notifications in real time.
- The server and monitor must handle the case where the FIFO is missing or the monitor is not running.
- The FIFO must be removed during cleanup if your program created it.

Example monitor output:

```
[NOTIFY] Ticket 1 created by Alex, priority HIGH
[NOTIFY] Ticket 2 created by Sara, priority MEDIUM
[NOTIFY] Ticket 1 closed
[NOTIFY] Server shutting down
```

8. Persistence and Logging

To make the system more realistic, the server must maintain simple files.

- **tickets.db:** The server must save ticket records to a text file. On startup, the server should load existing tickets if the file exists.
- **server.log:** The server must append important events such as client connect, client disconnect, ticket created, ticket closed, invalid command, save, and shutdown.
- **File safety:** The server must close files properly and avoid corrupting the ticket file during shutdown.

9. Graceful Shutdown and Cleanup

The server must shut down cleanly when the operator types `quit` or presses `Ctrl+C`.

- Notify connected clients, if possible, before closing their sockets.
- Write a final shutdown event to the FIFO notification channel.
- Save current tickets to tickets.db.
- Close all client sockets and the listening socket.
- Close FIFO and log file descriptors.
- Remove temporary socket/FIFO files if your design creates them.
- Exit without leaving background processes or temporary files behind.

10. Required Testing Demonstration

Your screenshots and video must demonstrate the following scenarios:

- Server starts successfully.
- ticketMonitor starts and waits for FIFO notifications.
- At least two clients connect to the server.
- Client 1 submits a ticket.
- Client 2 submits a different ticket.
- The monitor displays FIFO notifications for both tickets.
- A client lists open tickets.
- A client closes a ticket.
- The server operator runs show, stats, clients, and save from STDIN.
- The server rejects one invalid command safely.
- The server shuts down gracefully with quit or Ctrl+C.

11. Research and Out-of-the-Box Questions

Answer these questions in your WordDoc. These questions connect this assignment to real-world system design and current industry practices.

1. **IPC Selection in Industry:** Your system uses sockets for request/response and FIFO for notifications. Give one real-world example where this separation makes sense.
2. **Reliability:** What happens if the ticketMonitor is not running when the server writes to the FIFO? How would a production system avoid losing important events?
3. **Security:** What security risks exist if the server socket or FIFO is created under /tmp? Mention permissions, predictable names, spoofing, or unauthorized clients.
4. **Event Streaming Trend:** Modern systems often use event streams such as Kafka, RabbitMQ, Redis Streams, or cloud pub/sub services. How is your FIFO notification channel similar to and different from an event stream?

12. Deliverables

- ticketServer source file(s).
- ticketClient source file(s).
- ticketMonitor source file(s).
- Makefile with build, run, and clean targets.
- Explaining design decisions, socket model choice, protocol, FIFO design, testing steps, and research answers. (in WordDoc or video)
- Screenshots showing all required testing scenarios.
- A 2–10 minute demonstration/walkthrough video.

13. Suggested Repository Structure

```
assignment2/
├── Makefile
├── README.md
├── src/
│   ├── ticketServer.cpp
│   ├── ticketClient.cpp
│   ├── ticketMonitor.cpp
│   ├── Ticket.h
│   └── Ticket.cpp
├── screenshots/
└── sample-output/
```

14. Marking Rubric

Component	Marks	Does not meet expectations	Meets expectations	Exceeds expectations
Socket server and client	3	Server/client do not connect reliably.	Client and server communicate with required commands.	Robust connection handling, clear protocol, clean errors.
select() and multiple clients	3	No meaningful select() usage or only one client works.	Server uses select() for clients and STDIN.	Handles multiple clients cleanly and removes disconnected clients.
Ticket functionality and protocol	3	Missing major commands or unstable parsing.	SUBMIT, LIST, DETAIL, CLOSE, STATS, QUIT work.	Well-designed protocol with validation and useful responses.
FIFO notification monitor	2	FIFO missing or not functional.	Server sends notifications and monitor displays them.	Handles monitor absence, cleanup, and clear event formatting.
Persistence and server logging	2	No reliable file storage/logging.	Tickets saved/loaded and server events logged.	Clean file format, safe updates, readable logs.
Graceful shutdown and cleanup	2	Program hangs or leaves resources behind.	Ctrl+C/quit close sockets, FIFO, and files.	Shutdown notifies clients/monitor and handles edge cases.
Testing evidence and demo video	2	Minimal or unclear evidence.	Screenshots/video demonstrate required tests.	Video clearly explains design, code, challenges, and self-evaluation.
Research/reflection questions	2	Incomplete or superficial answers.	Most questions answered with reasonable explanations.	Strong real-world connections and critical comparison.
Code quality, Makefile, README	1	Difficult to build or poorly organized.	Builds cleanly with Makefile and includes README.	Clean structure, comments, consistent style, easy to reproduce.

Total: 20 marks

16. Academic Integrity and Use of AI Tools

This is an individual assignment unless your professor explicitly states otherwise. You may discuss concepts with classmates, but you must not share or copy implementation code.

If you use AI tools for explanation, debugging, or planning, you are still responsible for understanding, testing, and explaining your own code. Your video walkthrough and oral explanation may be used to verify authorship and understanding.

Assignment Submission:

- Complete all steps, Add all output-screenshot and explanations (if required) to a MS-Word file.

- Add the following declaration at the top of MSWORD file and source code

```
/******  
* DPS912-Assignment2  
* I declare that this lab is my own work in accordance with Seneca Academic Policy.  
* No part of this assignment has been copied manually or electronically from any other source  
* (including web sites) or distributed to other students.  
*  
* Name: _____ Student ID: _____ Date: _____  
*  
*  
*****/
```

- Please answer the following two declarations:

- **D1)** On a scale from 1 to 5, **How much did you use generative AI to complete this assignment?**
 - where:
 - **1** means you did not use generative AI at all
 - **2** means you used it very minimally
 - **3** means you used it moderately
 - **4** means you used it significantly
 - **5** means you relied on it almost entirely
 - **Your answer :**
- **D2)** On a scale from 1 to 5, **How confident are you in your understanding of the generative AI support you utilized in this assignment, and in your ability to explain it if questioned?**
 - where:
 - **1** means "Not confident at all – I do not understand the generative AI support I used and cannot explain it."
 - **2** means "Slightly confident – I understand a little, but I have many uncertainties."
 - **3** means "Moderately confident – I understand the majority of the support, though some parts are unclear."
 - **4** means "Very confident – I understand most of the AI support well and can explain it with minor gaps."
 - **5** means "Extremely confident – I fully understand the generative AI support I used and can clearly explain or justify it if asked."
 - **Your answer :**

- Please submit the Source code (zip all .c, .h, and makeFiles)

Important Note:

- **LATE SUBMISSIONS for labs.** There is a deduction of 10% for Late assignment submissions, and after three days it will grade of zero (0).
- This labs should be submitted along with a video-recording which contains a detailed walkthrough of solution. Without recording, the assignment can get a maximum of 1/3 of the total.

- Note: In case you are running out of time to record the video, you can submit the assignment (source code + screenshots) by the deadline and submit the video within 24 hours after the deadline.